PROJECT REPORT

Closed-Loop Dynamic QoS and Bandwidth Management for the 5G Core

A congestion-aware policy control loop between a simulated gNodeB and a mock PCF, inspired by the O-RAN RAN Intelligent Controller

Author	Arda Özgür
Programme	BSc Computer Engineering, Mudanya University
Supervisor	Dr. Öğr. Üyesi Nergis Erdem
Platform	IEEE 5G/6G Innovation Testbed
Project period	7 October 2026 – 19 October 2026
Report date	20 October 2026

Abstract

Static quality-of-service (QoS) rules cannot react when a cell approaches its radio capacity. This project builds a proof-of-concept closed loop that observes the Physical Resource Block (PRB) utilisation of a simulated 5G NR gNodeB (100 MHz, 273 PRBs, 30 kHz subcarrier spacing), compares it with a configurable congestion threshold of 80%, and, when the threshold is crossed, sends a 3GPP-style Npcf_SMPolicyControl request to a mock Policy Control Function (PCF) that throttles low-priority subscribers from 50/20 Mbps to 2/1 Mbps (downlink/uplink). The system consists of three containerised services, a simulator, a decision engine and a mock PCF, with all thresholds, target subscribers and QoS profiles kept in a YAML file.

The system was run on the IEEE 5G/6G Innovation Testbed and produced 720 logged samples. The detection and enforcement path worked: the engine selected the throttled profile from its first sample and made no redundant policy changes. However, the simulated load read 100% in 718 of 720 samples and never fell below the threshold, so the recovery path was not exercised. An offline replica of the simulator model shows why: with the chosen traffic parameters the unclipped demand exceeds capacity in more than 99.9% of draws. The report documents the design, the results, this limitation and a concrete plan to correct it.

Contents

1 Introduction
2 Background
3 System Design
4 Implementation
5 Experimental Setup
6 Results
7 Discussion and Limitations
8 Conclusion and Future Work
References
Appendix A Repository Map

1 Introduction

1.1 Motivation

In conventional cellular networks, QoS is largely configured per subscriber and stays fixed regardless of how busy the cell is. During peak events such as a stadium concert, a disaster or rush hour, the radio resources of a cell are exhausted and every user suffers. Modern O-RAN architectures address this with a RAN Intelligent Controller (RIC) that observes the radio network and adjusts behaviour in near real time. This project reproduces that idea at proof-of-concept scale: a software bridge watches the radio load and tells the core network to protect the cell by limiting best-effort traffic before the hardware limit is reached.

1.2 Objectives

1. Model gNodeB PRB utilisation from physical-layer quantities (SNR, fading, traffic demand).
2. Evaluate the load every 5 seconds against a configurable threshold.
3. Enforce QoS changes through a standards-inspired Npcf_SMPolicyControl request.
4. Keep thresholds, target subscribers and QoS profiles in configuration rather than code.
5. Provide a reproducible container deployment and a benchmark log with an automated plot.

1.3 Scope

In scope are the simulated radio-load model, the threshold-based decision logic, a mock PCF with a 3GPP-style REST endpoint, Docker Compose orchestration and benchmark logging. Out of scope are a real gNodeB and 5G Core, real user-plane rate enforcement, predictive or learning-based control, multi-cell and multi-slice scenarios, and production-grade security on the service-based interface.

2 Background

Physical Resource Block (PRB). The NR resource grid is allocated to users in PRBs of 12 subcarriers. With 30 kHz subcarrier spacing one PRB spans 360 kHz, and a 100 MHz carrier provides 273 PRBs. PRB utilisation is therefore a direct measure of how full the cell is.

5QI and MBR. The 5G QoS Identifier (5QI) selects a standardised QoS characteristic; 5QI 9 is the default non-guaranteed-bit-rate class used for best-effort internet traffic. The Maximum Bit Rate (MBR) caps the rate of a flow or session and is the lever used here to throttle subscribers.

Policy control. In the 5G Core, the Policy Control Function (PCF) provides session-management policy to the Session Management Function (SMF), which programmes the User Plane Function (UPF). The Npcf_SMPolicyControl service is the interface for this exchange [1, 2].

Closed-loop automation. O-RAN defines a RIC that monitors RAN metrics and applies control actions. The bridge engine in this project plays that role in a simplified form: observe, decide, act.

Capacity model. The simulator estimates per-user spectral efficiency with the Shannon bound, scaled by a practical factor to account for implementation losses [3].

3 System Design

Closed loop: radio load → decision → QoS enforcement

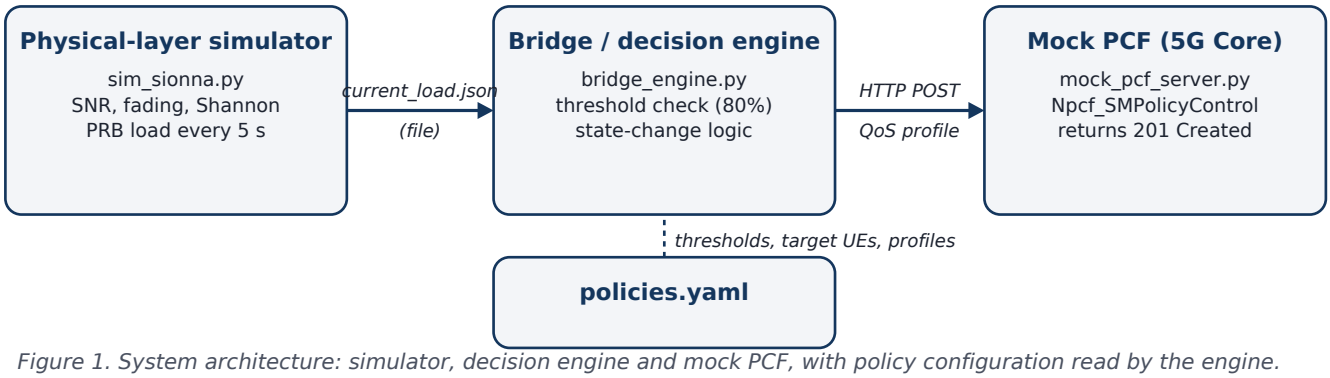


Figure 1. System architecture: simulator, decision engine and mock PCF, with policy configuration read by the engine.

Table 1. System components

Component	File	Responsibility
Physical-layer simulator	`sim_sionna.py`	Generates per-UE SNR, computes spectral efficiency and the PRB demand, writes `current_load.json` every 5 s
Decision engine	`bridge_engine.py`	Reads the load, chooses the `normal` or `throttled` profile, calls the PCF only when the profile changes, appends every sample to `benchmark_log.csv`
Mock PCF	`mock_pcf_server.py`	Flask service exposing `POST /npcf-smpolicy/v1/policies`; validates the payload and returns `201 Created`
Configuration	`policies.yaml`, `channel_config.json`	Threshold, target subscribers, QoS profiles; nominal channel and UE parameters
Orchestration	`docker-compose.yml`	Starts the three services on a shared bridge network with start-up ordering
Reporting	`benchmark_plotter.py`	Plots load over time, the threshold and the throttled samples

3.1 Decision logic

Every 5 seconds the engine reads `cell_load_percent`. If the value is at or above the threshold (80%) the target profile is `throttled`, otherwise `normal`. The PCF is contacted only when the target profile differs from the one currently applied, which avoids redundant signalling. Each sample is appended to the log regardless of whether a change occurred.

```
load = read(current_load.json)            # skip cycle if file unreadable
profile = "throttled" if load >= threshold else "normal"
if profile != applied_profile:
    for ue in target_ues:
        POST /npcf-smpolicy/v1/policies  {supi, trigger, qos_profile}
    applied_profile = profile
append(timestamp, load, profile) to benchmark_log.csv
```

3.2 Key design decisions

- **File-based interface** between simulator and engine keeps the proof of concept simple and decoupled; read errors from concurrent writes are tolerated by skipping the cycle.
- **State-change-only signalling** limits traffic towards the core.
- **Externalised policy** means VIP subscribers are protected simply by not listing them in `target_ues`.
- **Two-second HTTP timeout** prevents the control loop from blocking if the PCF is slow or unreachable.

4 Implementation

4.1 Radio-load model

For each active UE *i* the simulator draws a mean SNR uniformly between 0 and 25 dB and adds Gaussian fading with a standard deviation of 4 dB. Spectral efficiency, PRB capacity and cell load follow from:

```
SE_i   = 0.7 * log2(1 + SNR_i)              [bit/s/Hz]
C_i    = SE_i * 12 * 30 kHz                 [bit/s per PRB]
N_i    = demand_i / C_i                     [PRBs], demand_i ~ U(2, 12) Mbps
load   = min(100, 100 * sum(N_i) / 273)     [%]
```

The number of active UEs is drawn per cycle between 40 and 80 (the upper bound is exclusive in the code).

Table 2. Simulation parameters

Parameter	Value
Carrier / bandwidth	3.5 GHz, 100 MHz (C-band)
Subcarrier spacing / PRBs	30 kHz / 273
Scenario	Urban macro (UMa), per `channel_config.json`
Active UEs per cycle	40 – 80
Per-UE demand	2 – 12 Mbps
Update interval	5 s
Practical efficiency factor	0.7

4.2 QoS profiles

Table 3. QoS profiles in policies.yaml

Profile	5QI	MBR downlink	MBR uplink	Used when
`normal`	9	50 Mbps	20 Mbps	load below 80%
`throttled`	9	2 Mbps	1 Mbps	load at or above 80%
`critical_throttled`	9	512 kbps	256 kbps	defined, not yet used

The target subscribers are three SUPIs (`imsi-208930000000003`, `…04`, `…05`), representing a standard smartphone, a delay-tolerant IoT device and a best-effort video client.

4.3 PCF interface

```
POST /npcf-smpolicy/v1/policies
{ "supi": "imsi-208930000000003",
  "trigger": "CELL_CONGESTION",
  "policy_decision": { "qos_profile": {
      "5qi": 9, "maxbrDl": "2 Mbps", "maxbrUl": "1 Mbps" } } }
-> 201 Created {"status":"success"}
```

4.4 Deployment

`docker-compose.yml` defines three services on a shared bridge network: the mock PCF (Python 3.9 slim, Flask, port 8000), the simulator (official TensorFlow 2.15 image with Sionna) and the bridge engine, which starts after the other two. The project directory is mounted into each container so that the files exchanged between services are shared.

5 Experimental Setup

The full stack was run on the IEEE 5G/6G Innovation Testbed. The decision engine logged one line per cycle (timestamp, load percentage, active policy) to `benchmark_log.csv`, and `benchmark_plotter.py` produced the benchmark graph. The evaluation questions were: (a) does the engine detect congestion and apply the throttled profile, (b) does it avoid redundant policy changes, and (c) does it restore the normal profile when the load falls.

Table 4. Benchmark run

Item	Value
Samples logged	720
Sampling interval	about 5 s (median 5.03 s)
First / last sample	18 Oct 2026 11:59 / 20 Oct 2026 12:23 (log timestamps)
Active sampling	about 63 min (first session) + about 7 min (after a pause of about 47 h)
Short pauses	two of about 80 s inside the first session

6 Results

Table 5. Summary of the testbed log

Metric	Observed
Samples at 100% load	718 of 720 (99.7%)
Mean load	99.98%
Minimum load	90.75% (at about 3,793 s)
Other dip	93.99% (at about 946 s)
Samples below 80% threshold	0
Samples with `throttled` policy	720 of 720
Policy changes after the first sample	0

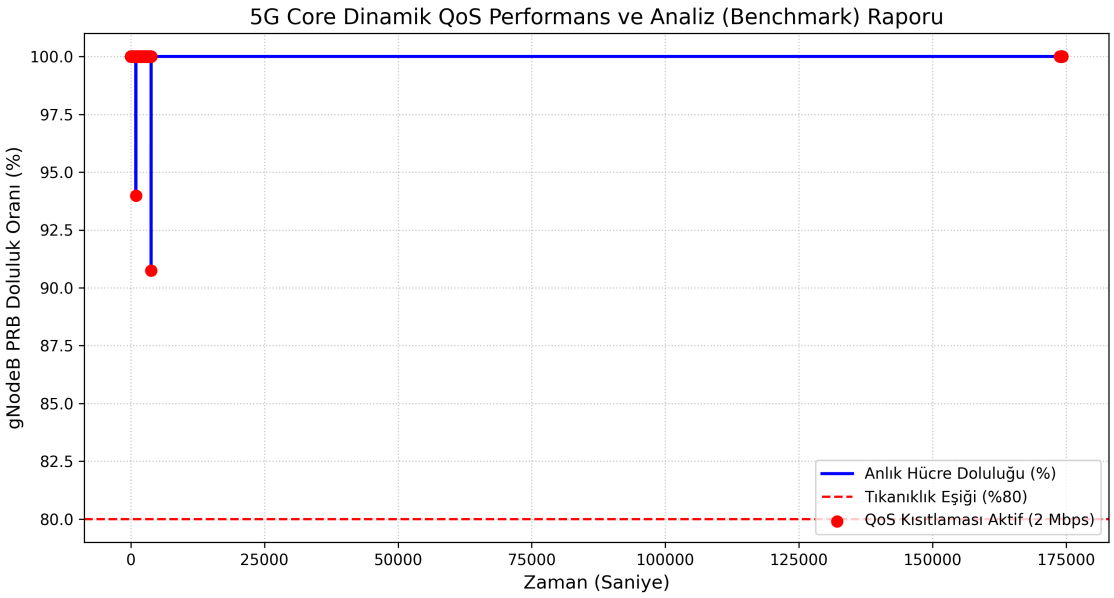


Figure 2. Benchmark graph produced by benchmark_plotter.py. The time axis spans about 174,000 s because of the idle gap, which compresses the data at the left edge.

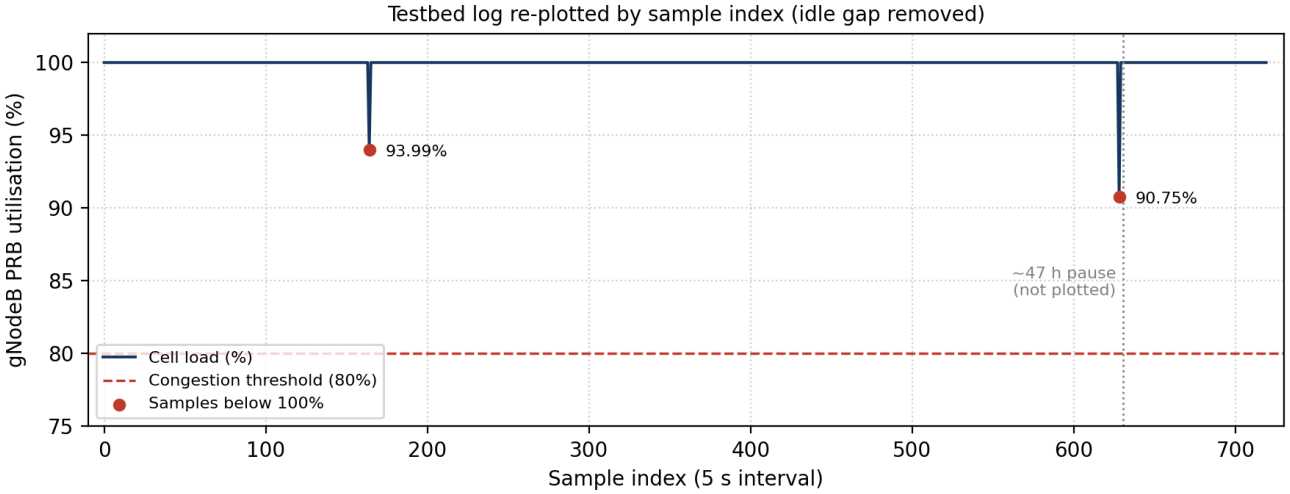


Figure 3. The same log plotted by sample index with the idle gap removed (generated from benchmark_log.csv).

Detection and enforcement. The engine selected the throttled profile on its first sample and kept it for the whole run, with no policy changes after that. The log does not record PCF calls, so the number of requests sent is inferred from the state-change logic rather than measured.

Recovery. Because the load never dropped below 80%, the transition from throttled back to normal never occurred, and the benefit of throttling on cell load could not be demonstrated.

7 Discussion and Limitations

7.1 Why the load saturates

To check whether the saturation is a property of the model rather than of the testbed run, the simulator equations were replicated offline and sampled 20,000 times. The unclipped load had a median of about 232% (5th to 95th percentile: about 143% to 343%), exceeded 100% in 99.94% of draws and never fell below 80% (minimum about 92%). In other words, with 40–80 UEs each asking for 2–12 Mbps, the modelled demand is roughly two to three times the capacity of a 100 MHz cell, so the congestion threshold is crossed on essentially every cycle. The two dips seen in the log are consistent with the rare low-demand draws of this model.

7.2 Limitations

1. **Open loop in practice.** The throttling decision is sent to the PCF but never feeds back into the simulator, so the load does not respond to the control action.
2. **Traffic calibration.** The demand parameters leave no operating range between 50% and 100% load, so the recovery path cannot occur.
3. **Single threshold without hysteresis.** A load hovering near 80% could make the profile flap.
4. **Simulation fidelity.** The channel is a statistical model. The values in `channel_config.json` are not read by the simulator, and Sionna's full channel models are not used.
5. **Mocked enforcement.** The mock PCF reports that the policy reached the UPF but applies no rate limit.
6. **Plotting.** The plotter measures time from the first sample, which distorts the graph when the run is interrupted.

These limitations mean the results validate the detection and enforcement logic and the tooling, not the effectiveness of throttling.

8 Conclusion and Future Work

The project delivers a working, containerised closed loop from radio-load observation to a standards-style policy request, with configuration-driven behaviour and an automated benchmark pipeline. The testbed run confirmed correct detection and state handling, and also exposed a calibration problem that prevents the recovery path from being tested. Correcting it is the first step of the future work below.

Table 6. Improvement plan

Priority	Improvement	Expected benefit
High	Feed the throttle back into the simulator by capping the demand of target UEs	Truly closed loop
High	Recalibrate demand and UE counts for a 50–100% load range; add ramp, spike and plateau scenarios	Exercises both transitions
High	Add hysteresis (for example throttle at 80%, restore at 70%)	Prevents flapping
Medium	Use the `critical_throttled` profile above 95% load	Multi-level control
Medium	Replace the mock PCF with open5GS or free5GC	Real-core validation
Medium	Unit tests for the decision logic; plot by sample index or handle gaps	Quality
Low	Message bus instead of file exchange; TLS/OAuth2 on the interface; ML-based load prediction	Production realism, research

References

[1] 3GPP TS 23.503, Policy and charging control framework for the 5G System (5GS); Stage 2.

[2] 3GPP TS 29.512, 5G System; Session Management Policy Control Service; Stage 3.

[3] J. Hoydis et al., “Sionna: An Open-Source Library for Next-Generation Physical Layer Research,” arXiv:2203.11854, 2022.

[4] O-RAN Alliance, O-RAN architecture description and RIC specifications (www.o-ran.org).

Appendix A Repository Map

File	Purpose
`sim_sionna.py`	Physical-layer load simulator
`channel_config.json`	Channel and UE parameters (nominal)
`current_load.json`	Simulator output read by the engine
`bridge_engine.py`	Decision engine and PCF client
`policies.yaml`	Thresholds, target UEs, QoS profiles
`mock_pcf_server.py`	Mock 5G Core PCF (Flask)
`docker-compose.yml`	Service orchestration
`requirements.txt`	Python dependencies
`benchmark_log.csv`	Per-cycle log: timestamp, load, policy
`benchmark_plotter.py`	Report graph generator
`README.md`	Project documentation